Navegação

Navegação anônima

Cristiano Rodrigues

O que vamos aprender nessa aula?

- Navigator
- MainAxisAlignment
- CrossAxisAlignment

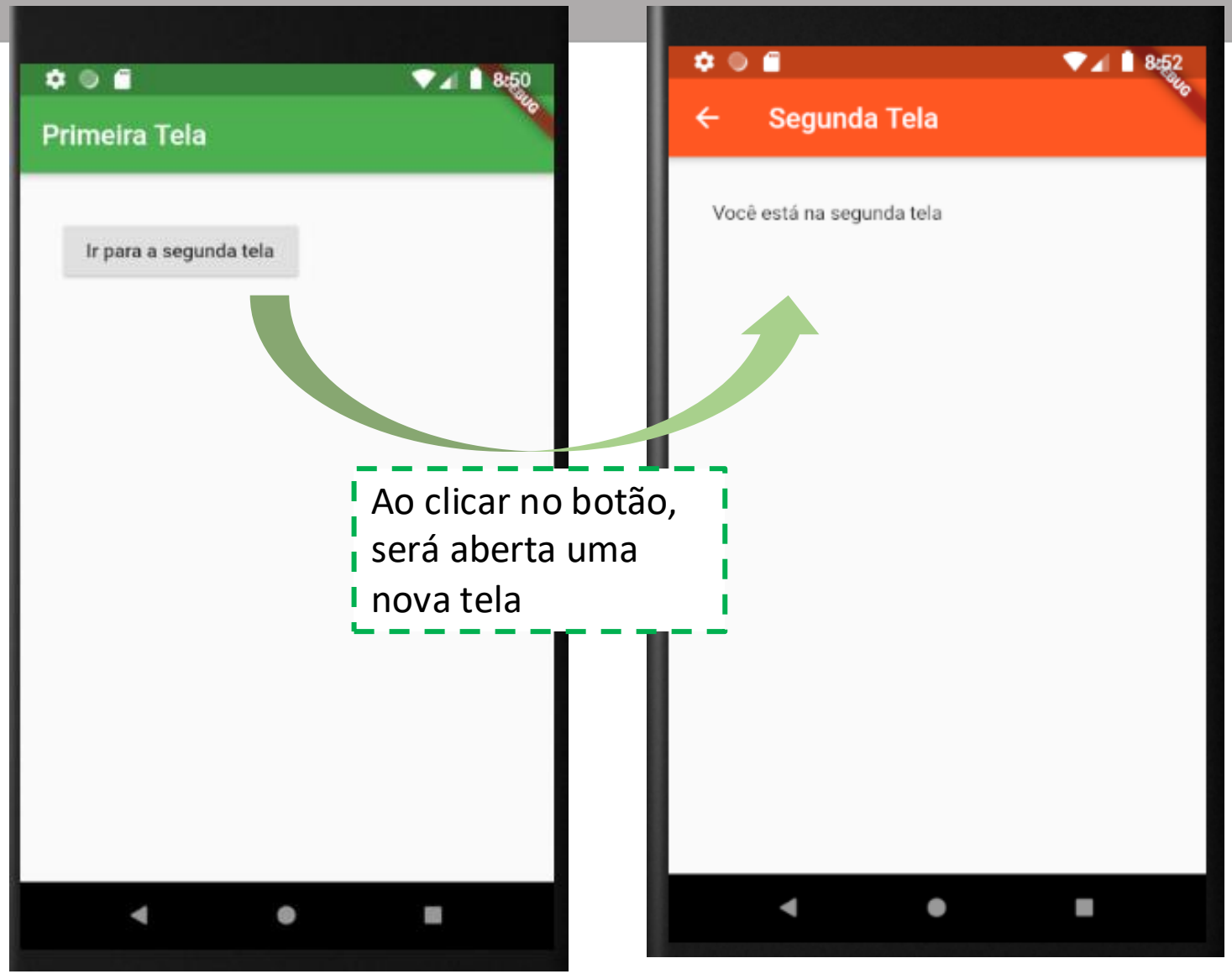
Navigator

- Classe responsável por fazer a navegação entre uma tela e outra
- O método **push()** abre uma nova tela **sem fechar** a anterior (contexto)
- O método **pop()** abre uma nova tela fechando a anterior (contexto)
- Ao usar **push()** e **pop()**, podemos chamar uma página específica ou uma rota previamente nomeada.

Navigator

No exemplo, teremos duas telas: a primeira tela e a segunda.

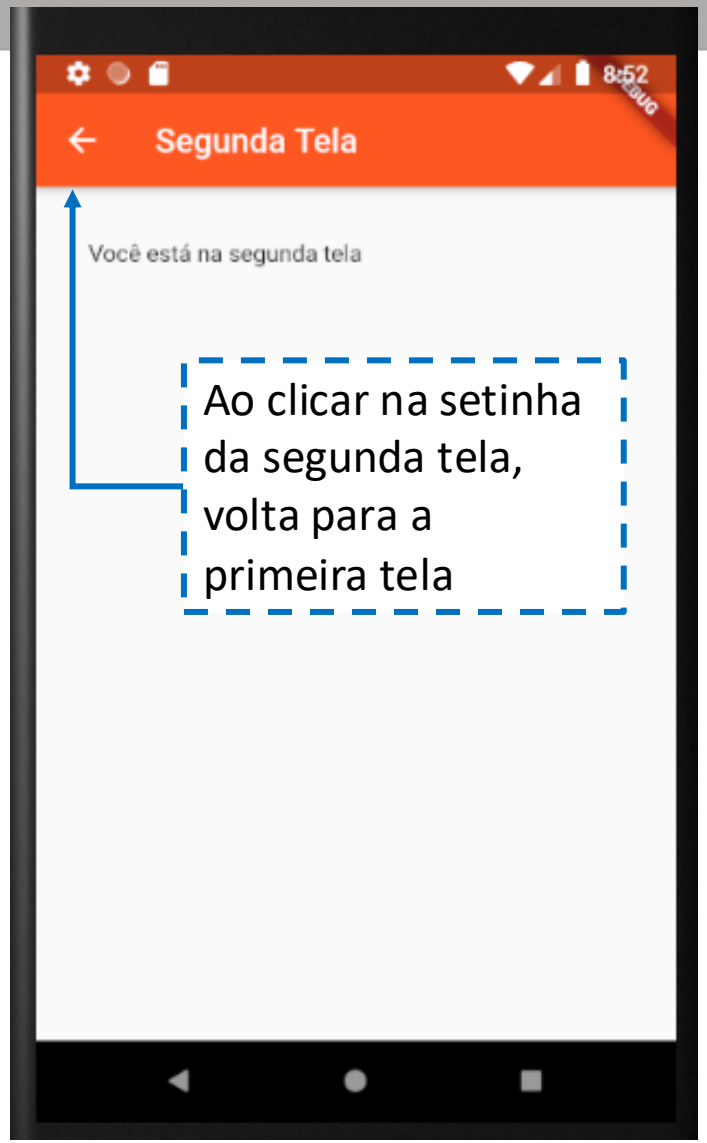
Na primeira teremos um botão, que ao clicar abre a segunda tela



Navigator

A segunda tela ficará sobreposta à primeira.

Portanto, a Primeira Tela não será fechada, mas ao voltar usando a setinha a Segunda Tela é fechada



Navigator

Para este estudo dirigido, crie duas classes, uma para cada tela, e use *extends StatelessWidget* para que ambas sejam widgets. Configure-as como achar melhor e adicione um botão em cada uma.

A seguir veremos como configurar cada botão para adicionar uma rota.

Navigator

Na opção `onPressed` do botão, use um `Navigator` com `push`, como mostrado abaixo:

```
FilledButton(
    child: Text('Ir para segunda tela'),
    onPressed: (){
      Navigator.of(context).push(route);
  },
 )
```

Context (contexto) é a tela na qual estamos. Como já temos no método build o context, basta colocar context mesmo aqui no **push()**

Definir a **rota**. Existe mais de uma forma para fazer. Vamos usar o método mais fácil que é usando o **MaterialPageRoute**

Navigator

```
FilledButton(
    child: Text('Ir para segunda tela'),
    onPressed: (){
     Navigator.of(context).push(
       MaterialPageRoute(
         builder: (context) => SegundaTela()
       )
     );
   },
 )
```

Usamos o **MaterialPageRoute**, que é uma classe que já tem tudo que precisamos para definir a rota

Argumento **builder**, com uma função anônima, precisa de um parâmetro que é o contexto

Indicamos qual é Tela para a qual devemos ir, de forma simples, indicando o nome da classe

Ao usarmos **push()**, empilhamos a segunda tela sobre a primeira. Portanto, o estado da primeira tela é mantido, mas sua visibilidade fica ocultada pela segunda tela.

Navigator

```
FilledButton(
    child: Text('Ir para segunda tela'),
    onPressed: (){
     Navigator.of(context).pushReplacement(
       MaterialPageRoute(
         builder: (context) => SegundaTela()
       )
     );
   },
 )
```

Caso não queiramos que tenha opção para retornar à tela anterior, basta usar o **pushReplacement()**

Navigator

```
FilledButton(
    child: Text('Ir para terceira tela'),
    onPressed: (){
          Navigator.of(context).pop();
          …
      );
  },
 )
```

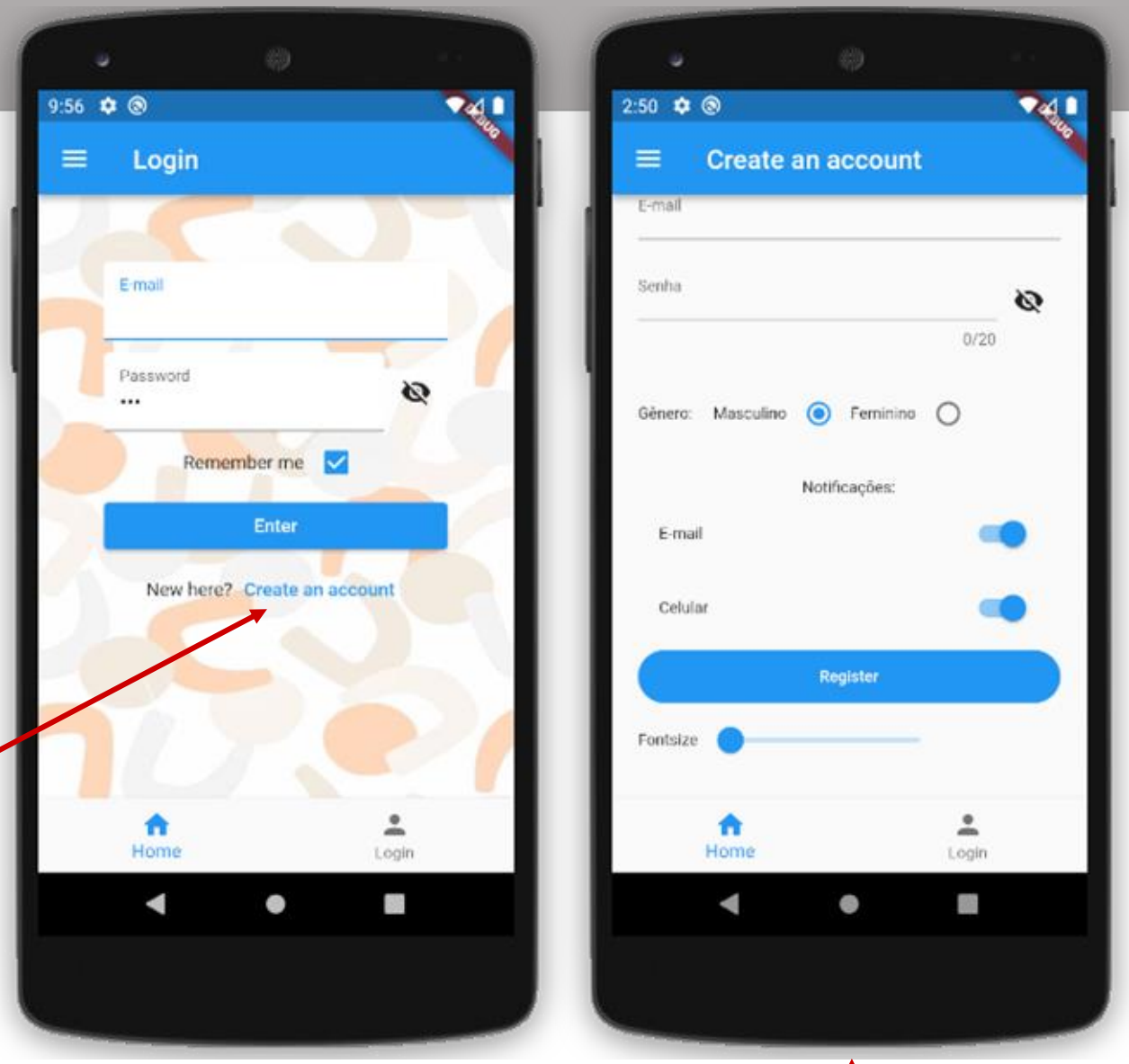
Caso queiramos remover a última tela da nossa pilha de telas, podemos usar o **pop()**.

O método **pop()** remove a tela atual da pilha de navegação e volta para a tela anterior.

Prática - Entregar

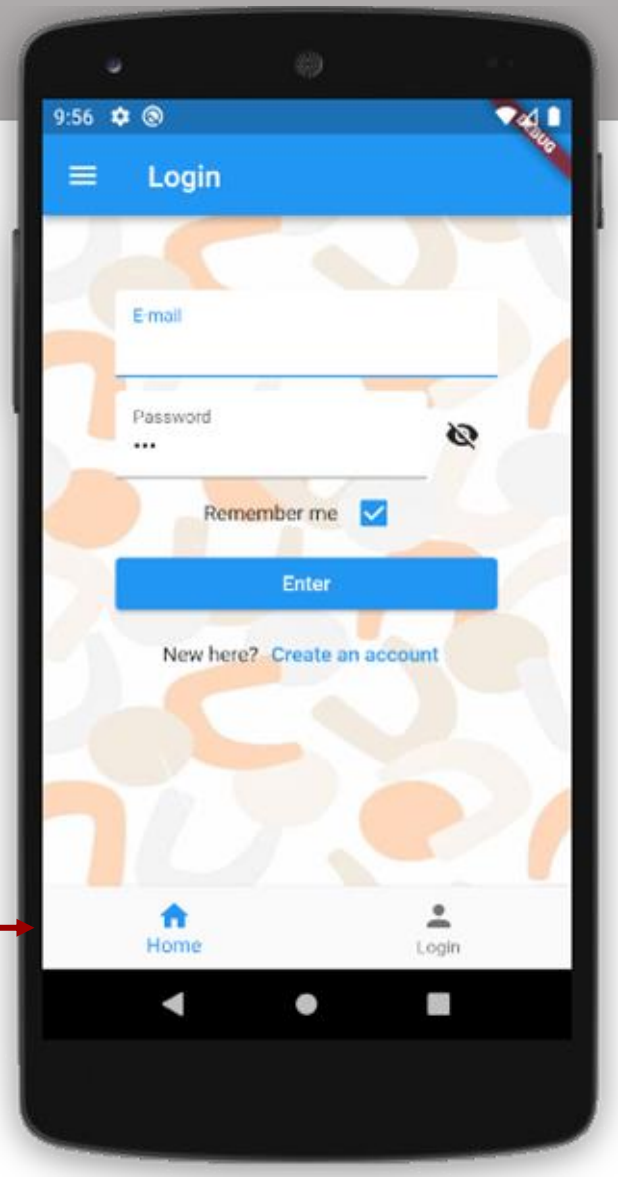
- Use as telas de login e cadastro criadas anteriormente
- Utilize **Navigator** para ir da tela de login para a de cadastro ao clicar em “Create an account”.

Ao clicar em “Create an account”, a tela de cadastro é aberta



Prática - Entregar

- A navegação entre páginas, também pode ser feita usando widgets de barra de navegação.
- Já vimos como criar um menu na barra inferior do app usando **bottomNavigationBar**, mas nosso menu ainda está estático.
- Para este exercício, crie três páginas: uma MainPage (StatefulWidget), uma HomePage e uma AboutPage (ambas StatelessWidget).



Exemplo de menu na barra inferior

Prática - Entregar

- Na MainPage (StatefulWidget) adicione um **MaterialApp** e, em seu *body*, adicione um **Scaffold**.
- Para esse exercício nossa aplicação terá somente um **Scaffold**.
- No *Scaffold*, crie um **BottomBavigationBar** como abaixo.

```
bottomNavigationBar: BottomNavigationBar(
   currentIndex: _currentIndex,
   items: <BottomNavigationBarItem>[...],
   onTap: (index){...});
 },
)
```

Prática - Entregar

- No parâmetro `items`, deve-se informar uma lista de `BottomNavigationBarItem`. Essa lista indica os ícones e texto que aparecerão na nossa barra de navegação.
- Só teremos duas outras páginas, HomePage e AboutPage. Criamos uma opção para cada

```
bottomNavigationBar: BottomNavigationBar(
    currentIndex: _currentIndex,
    items: <BottomNavigationBarItem>[
        BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.account_balance), label: 'About')
    ],
    onTap: (index){...});
 },
)
```

Prática - Entregar

- No parâmetro `currentIndex`, deve-se informar um valor inteiro indicando o item (da lista `items`) que será mostrado como ativo.
- Como isso dependerá do click do usuário, criamos uma variável **`_currentIndex`** , que inicialmente terá valor 0
  - Em **_MainPageState**, adicionar `int _currentIndex = 0;`

```
bottomNavigationBar: BottomNavigationBar(
    currentIndex: _currentIndex,
    items: <BottomNavigationBarItem>[
        BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.account_balance), label: 'About')
    ],
    onTap: (index){...});
 },
)
```

Prática - Entregar

- Para que o valor de **_currentIndex** seja alterado conforme o item que o usuário clicar, devemos alterar seu valor em **onTap**, que indica as mudanças que ocorrerão sempre que o usuário clicar em um ícone do BottomNavigationBar.
- Ao alterar o valor de uma variável estamos alterando o estado do widget. Portanto, devemos fazer essa alteração dentro de uma função setState

```
bottomNavigationBar: BottomNavigationBar(
   currentIndex: _currentIndex,
   items: <BottomNavigationBarItem>[...],
   onTap: (index){
    setState(() {
      _currentIndex = index;
    });
   },
)
```

Prática - Entregar

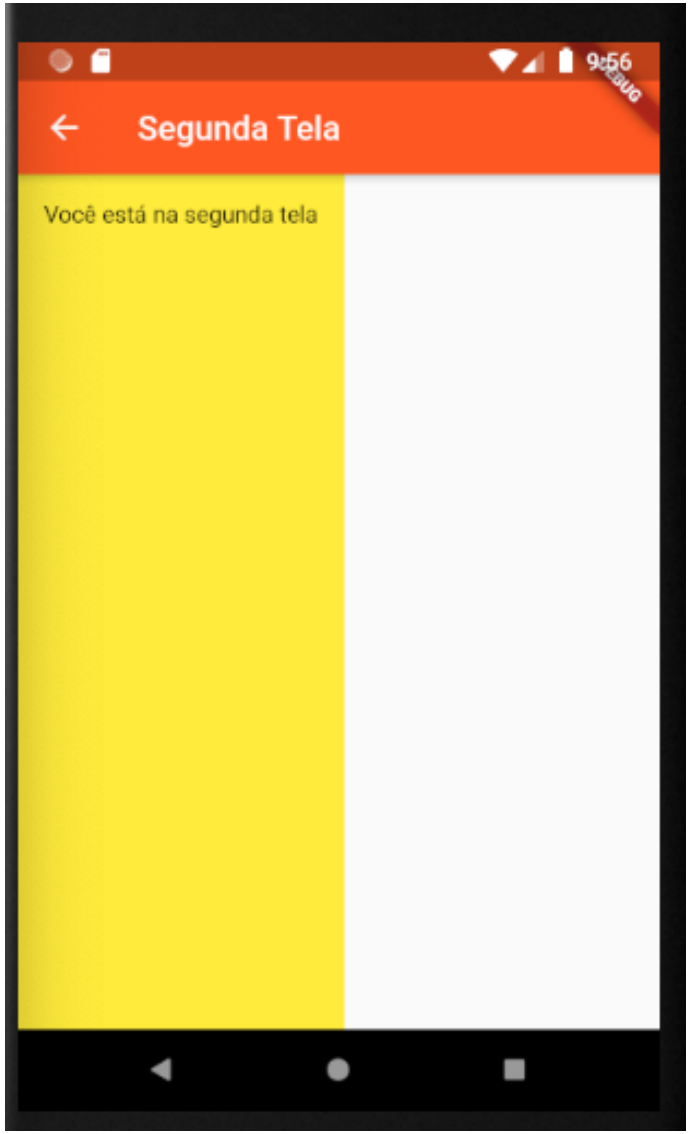
- Com essas mudanças nosso `BottomNavigationBar` está pronto, mas sem mudança de página.
- Para alterar a página conforme o valor em **`_currentIndex`**, usaremos o widget `IndexedStack`, que permite declarar uma lista de widgets e indicar o índice do widget atual.
- Criaremos o `IndexedStack` no *body* do Scaffold. Seu índice será indicado pelo valor na variável **`_currentIndex`** e a lista de widgets que ele controlará terá as páginas `HomePage` e `AboutPage`, criadas anteriormente.
- Agora seu aplicativo mudará de página conforme o ícone ativo do menu inferior.

```
body: IndexedStack(index: _currentIndex, children: [HomePage(), AboutPage()]),
```

MainAxisAlignment

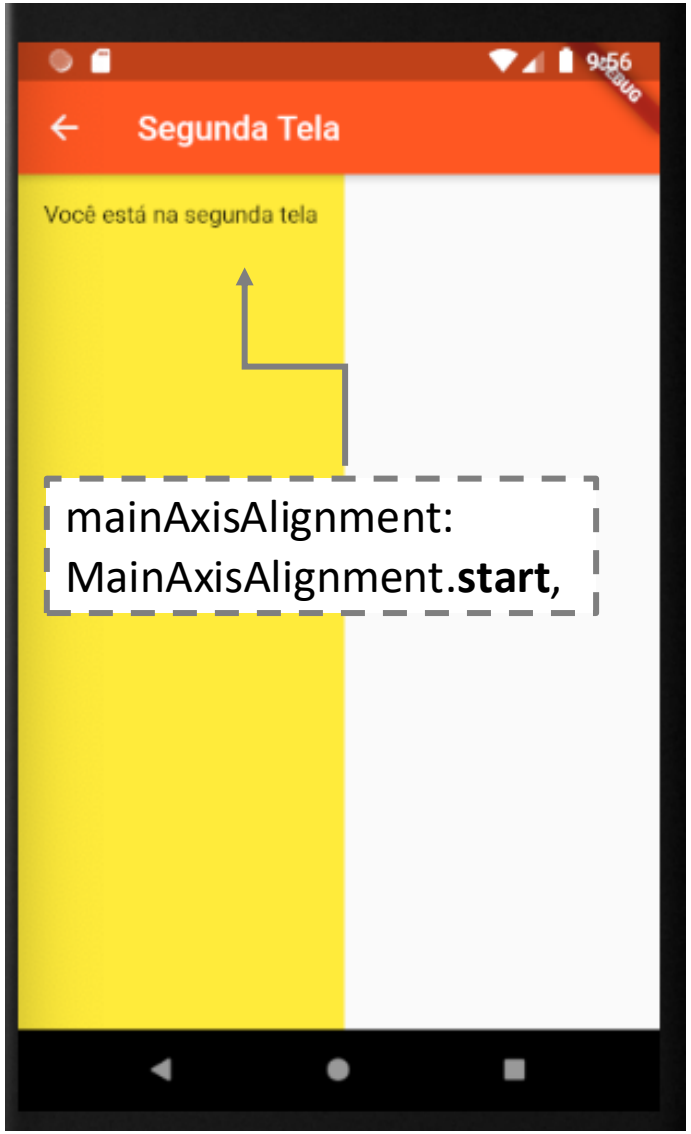
- Permite a movimentação de botões, textos, etc no eixo principal de uma linha (**Row**) ou de uma coluna (**Column**)
- É possível usar MainAxisAlignment.start (início), MainAxisAlignment.center (centro) e MainAxisAlignment.end (fim) da linha ou coluna, sendo o padrão o start

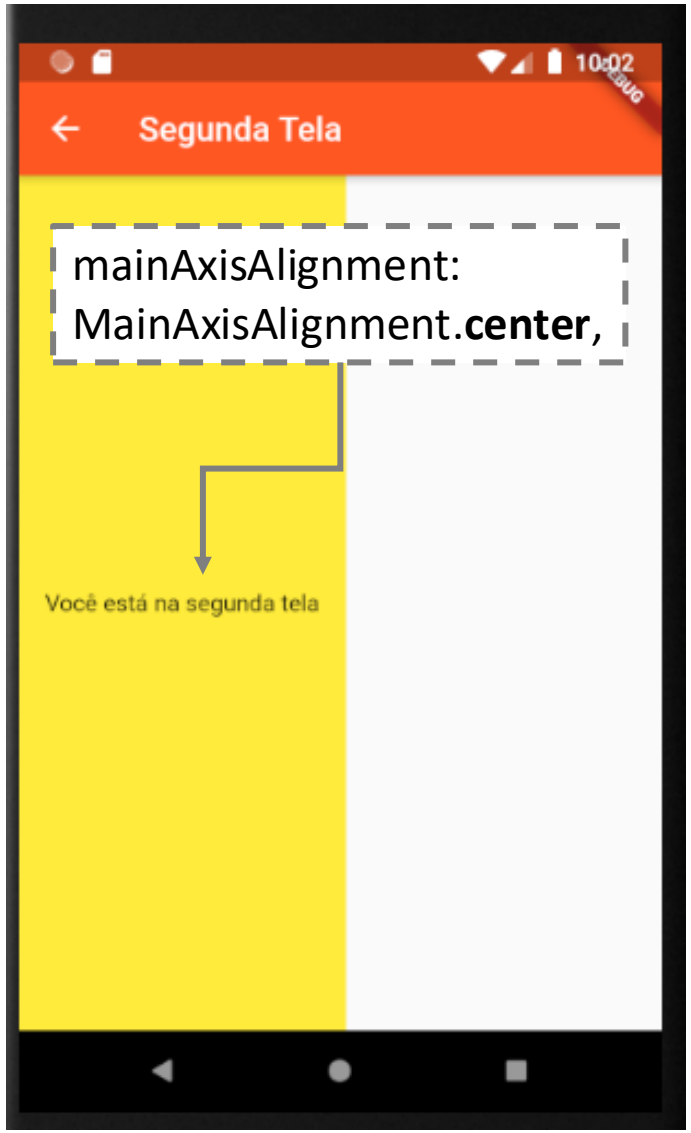
MainAxisAlignment

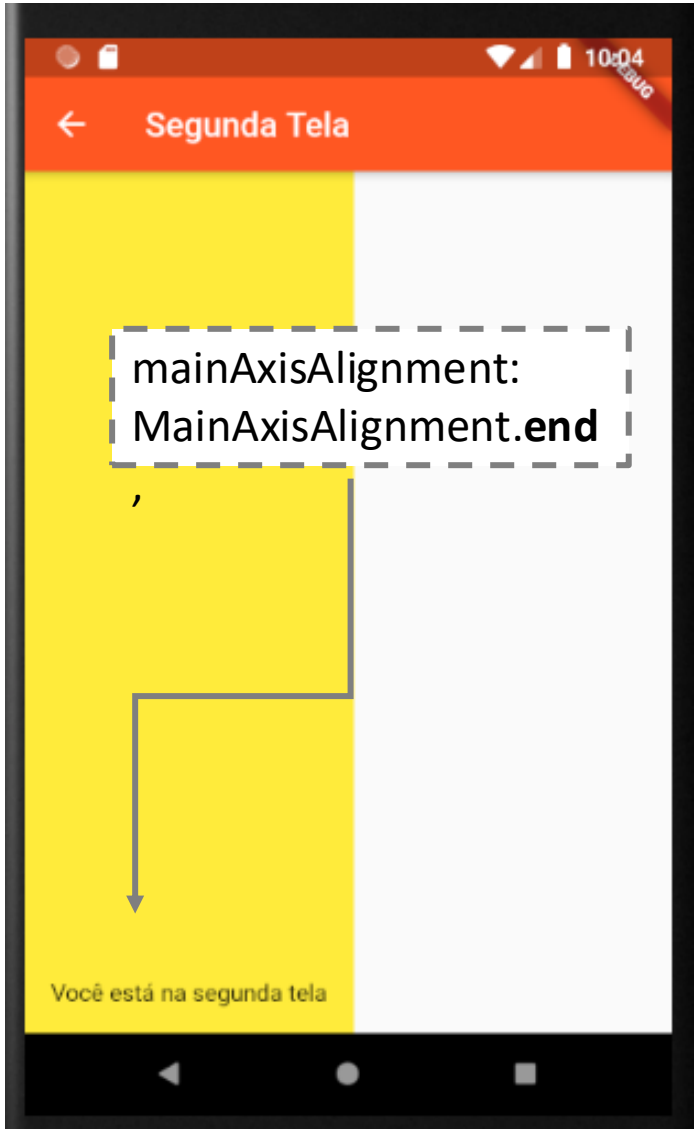



```
body: Container(
  color: Colors.yellow,
  padding: EdgeInsets.all(16),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.start,
    children: <Widget>[
      Text("Você está na segunda tela")
    ], // <Widget>[]
  ), // Column
), // Container
```

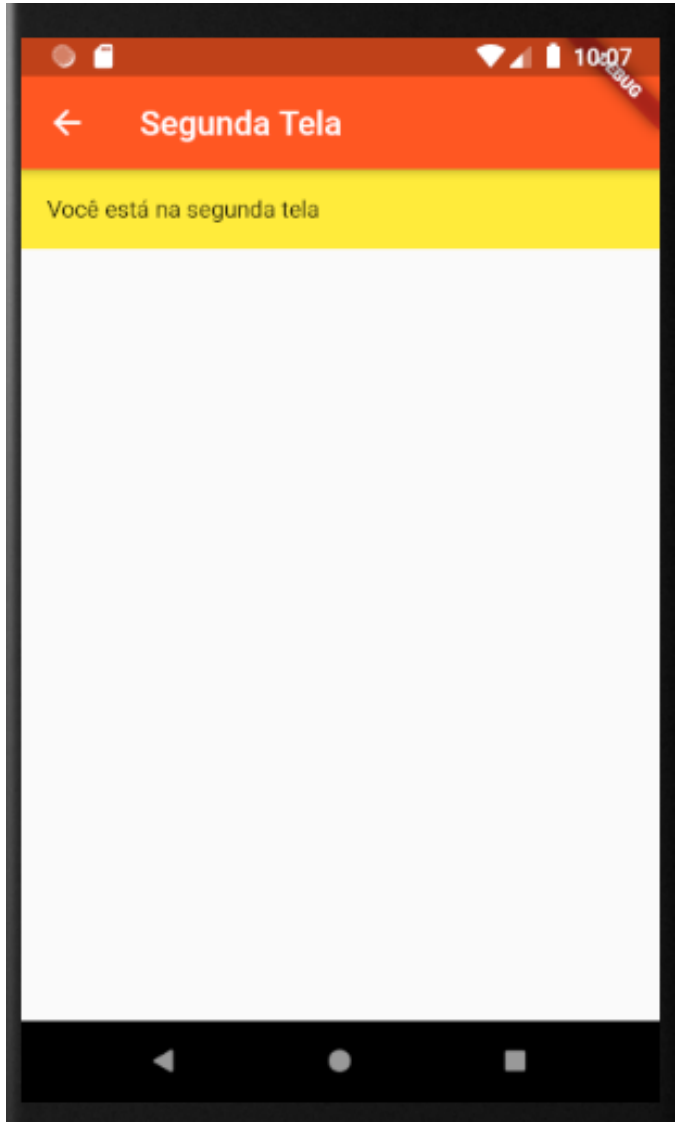
MainAxisAlignment





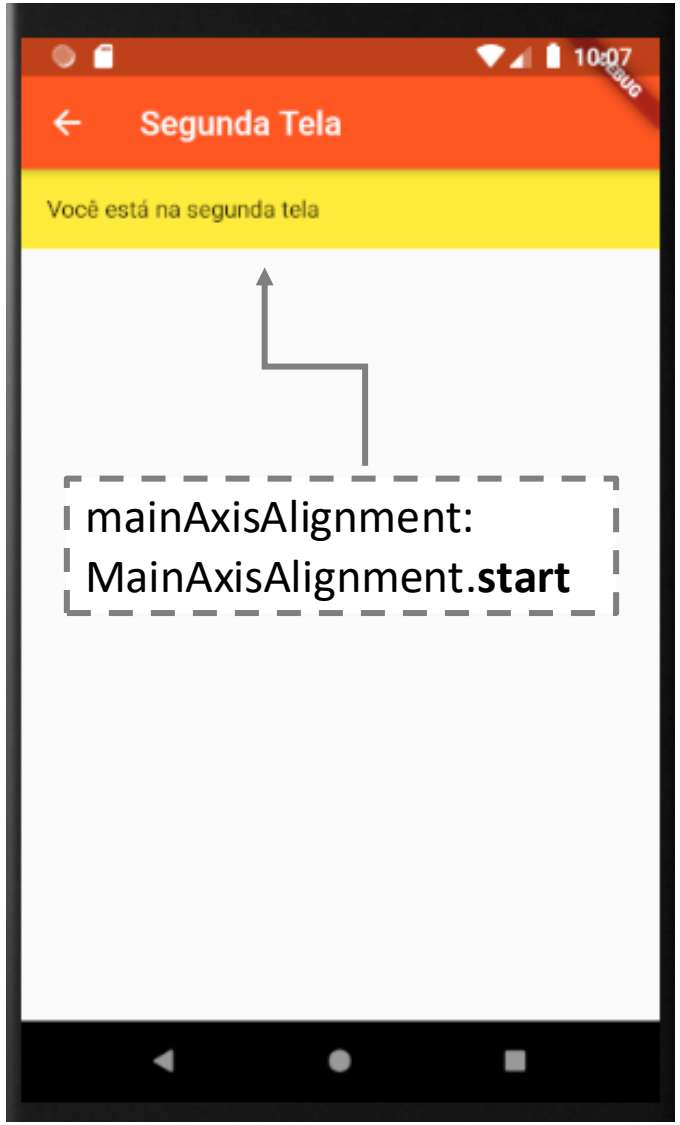


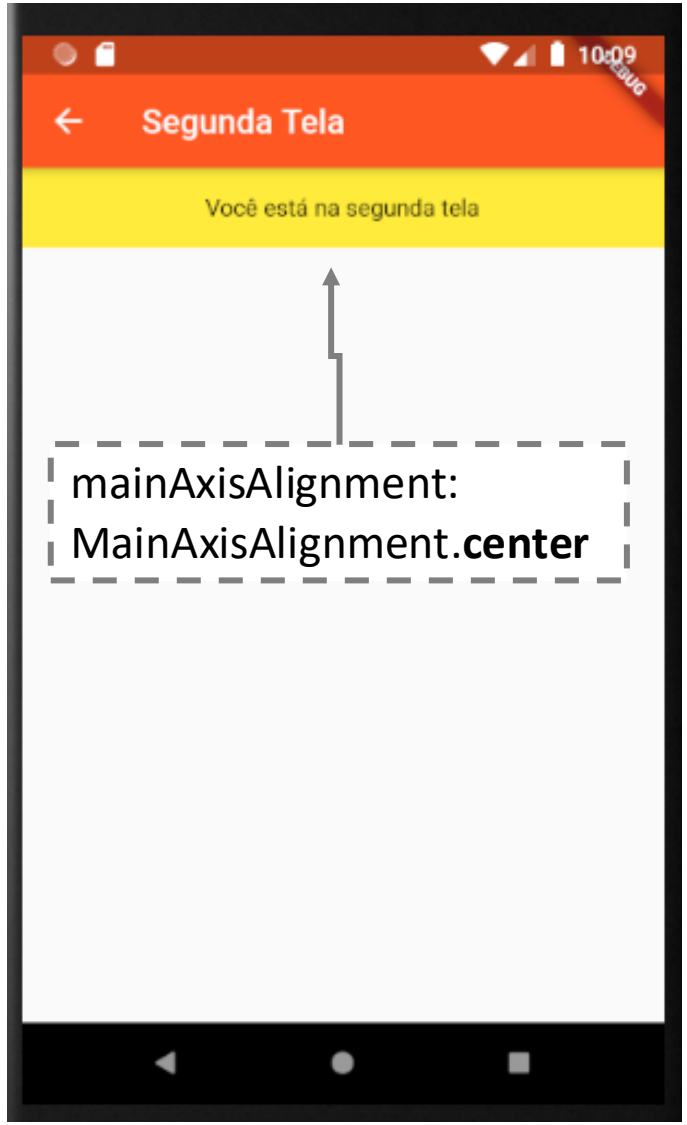
MainAxisAlignment

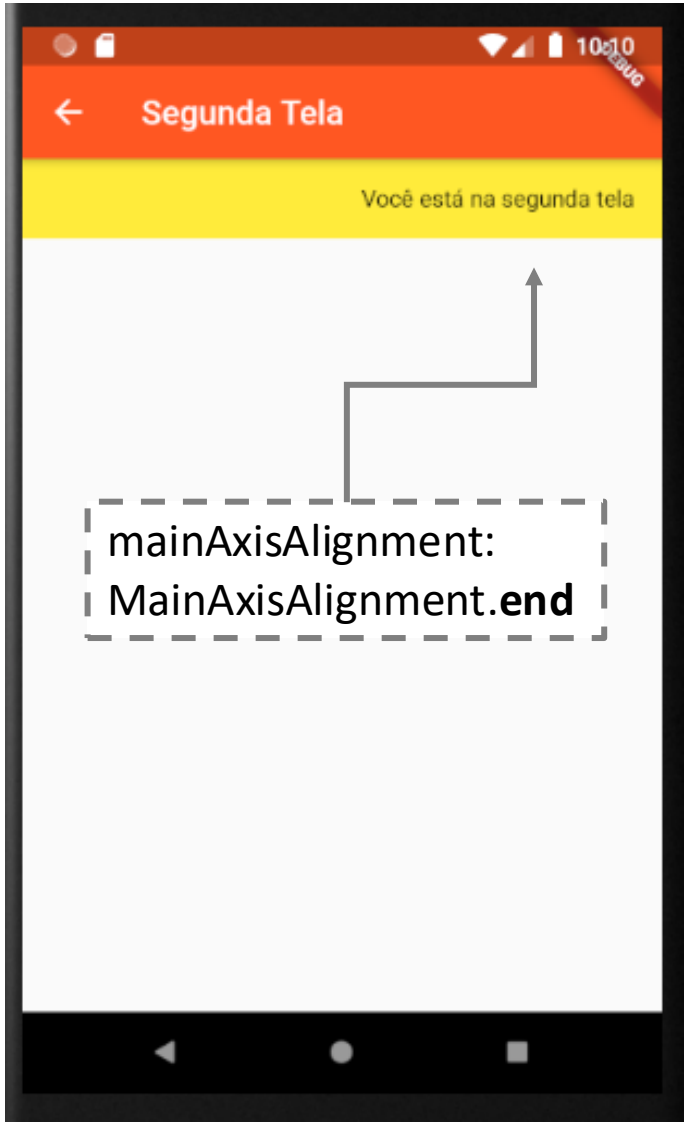



```
body: Container(
  color: Colors.yellow,
  padding: EdgeInsets.all(16),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.start,
    children: <Widget>[
      Text("Você está na segunda tela")
    ], // <Widget>[]
  ), // Row
), // Container
```

MainAxisAlignment



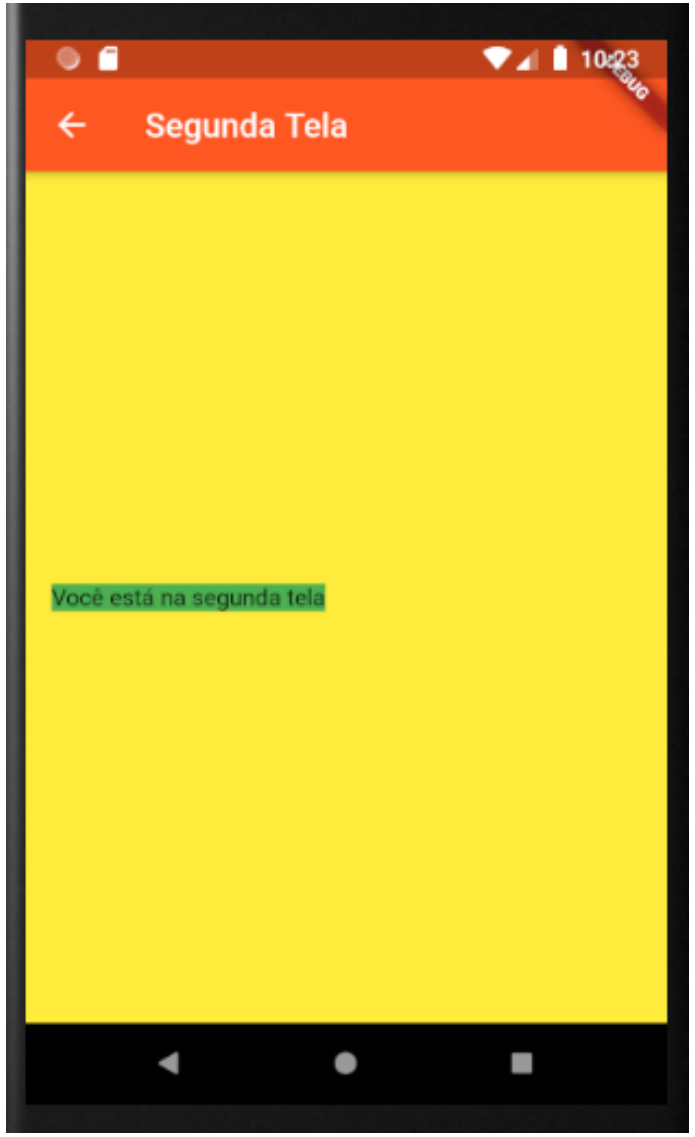




CrossAxisAlignment

- Permite a movimentação de botões, textos, etc no eixo cruzado de uma linha ou de uma coluna
- O padrão do **CrossAxisAlignment** é o centro (center)
- É importante lembrar de estender a largura (no caso da coluna) ou altura (no caso da linha) do container

CrossAxisAlignment

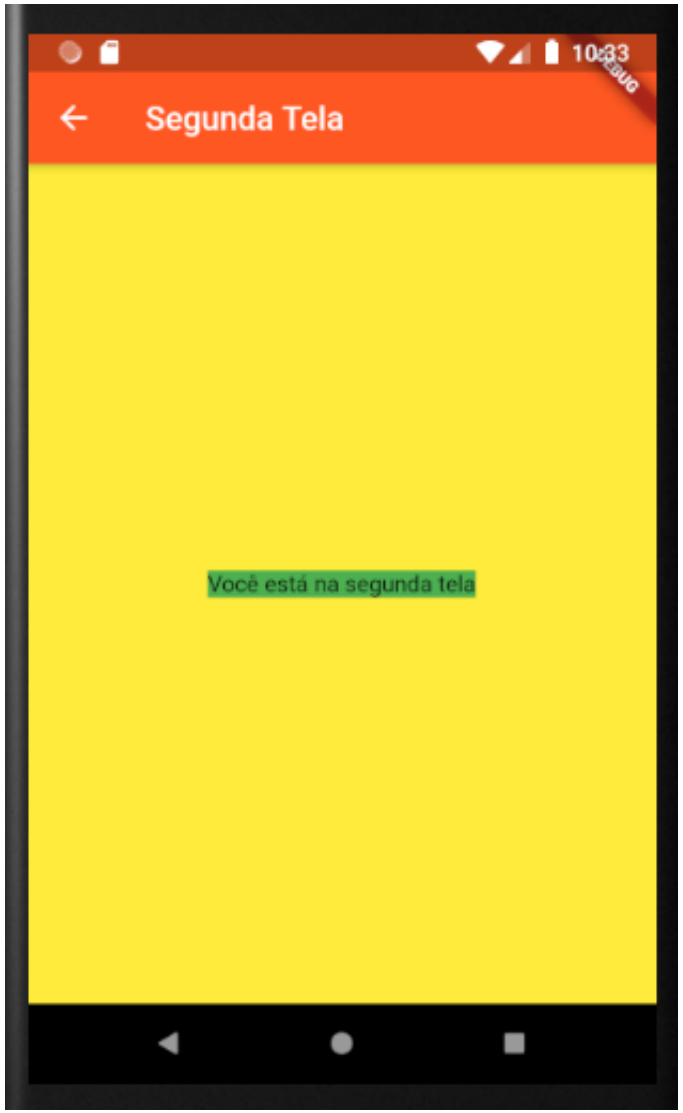



```
body: Container(
  height: double.infinity,
  color: Colors.yellow,
  padding: EdgeInsets.all(16),
  child: Row(
    crossAxisAlignment: CrossAxisAlignment.center,
    children: <Widget>[
      Text("Você está na segunda tela",
        style: TextStyle(
          backgroundColor: Colors.green,
        ),) // TextStyle, Text
    ], // <Widget>[]
  ), // Row
), // Container
```

Dessa forma, a altura do container será “infinita”, ou seja, considerando a tela toda.

Ficou no centro porém no canto, pois por padrão o **MainAxisAlignment** é start

CrossAxisAlignment




```
body: Container(
  height: double.infinity,
  color: Colors.yellow,
  padding: EdgeInsets.all(16),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.center,
    crossAxisAlignment: CrossAxisAlignment.center,
    children: <Widget>[
      Text("Você está na segunda tela",
        style: TextStyle(
          backgroundColor: Colors.green,
        ),) // TextStyle, Text
    ], // <Widget>[]
  ), // Row
), // Container
```

Agora o texto ficou no centro tanto em termos de eixo cruzado, como também de eixo principal

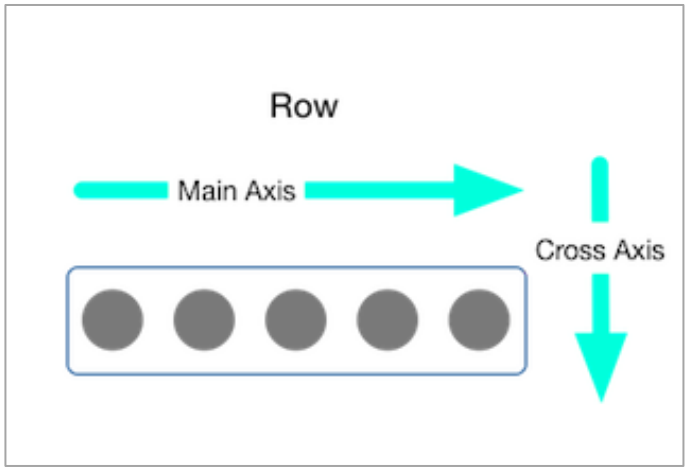
CrossAxisAlignment

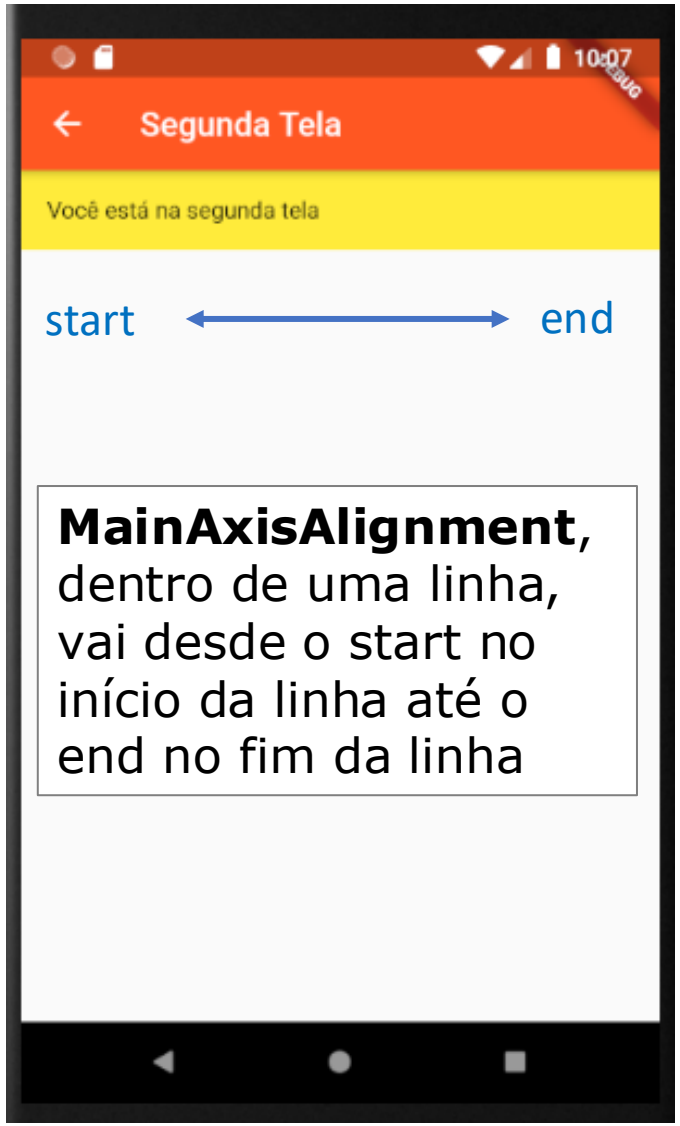
```
body: Container(
  width: double.infinity,
  color: Colors.yellow,
  padding: EdgeInsets.all(16),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.center,
    children: <Widget>[
      Text("Você está na segunda tela",
        style: TextStyle(
          backgroundColor: Colors.green,
        ),) // TextStyle, Text
    ], // <Widget>[]
  ), // Column
), // Container
```

Dessa forma, a largura do container será “infinita”, ou seja, considerando a tela toda.

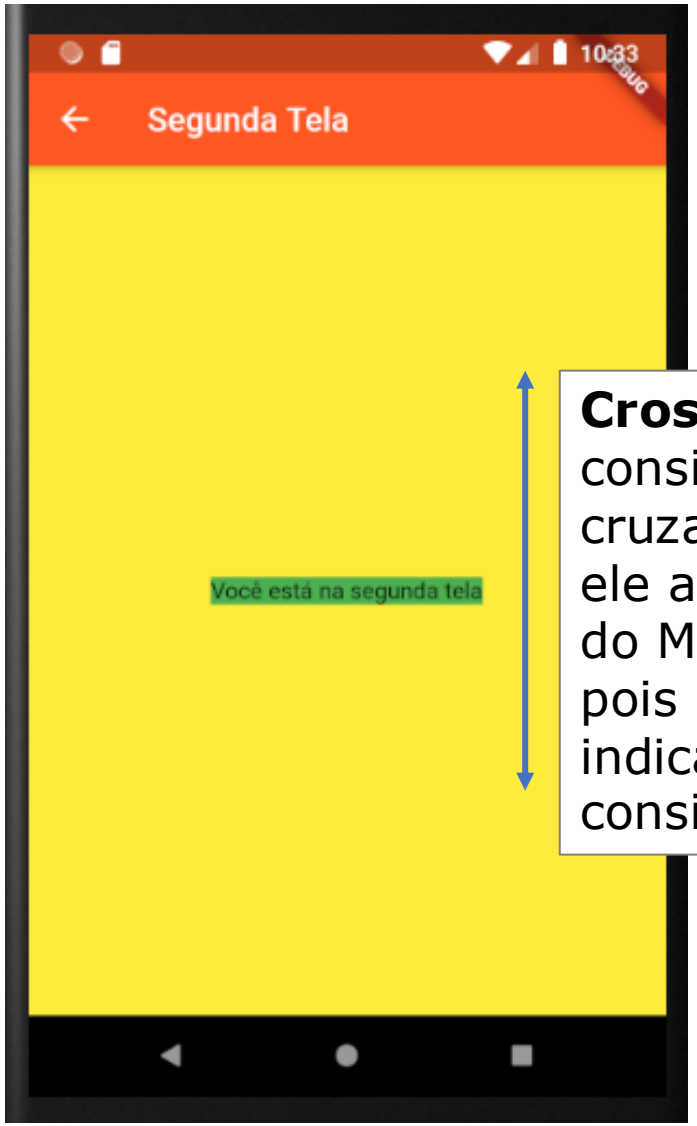
Ficou no centro porém no topo, pois por padrão o **MainAxisAlignment** é start, e agora estamos considerando a **coluna**

MainAxisAlignment *X* CrossAxisAlignment - Row





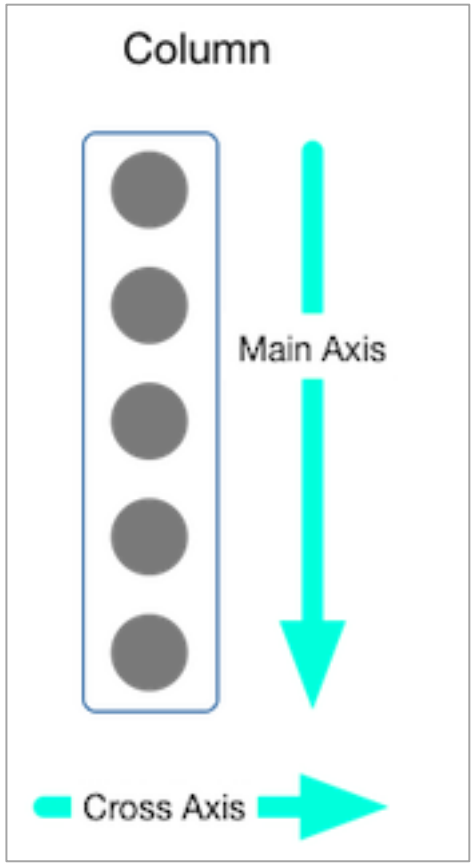
MainAxisAlignment, dentro de uma linha, vai desde o start no início da linha até o end no fim da linha

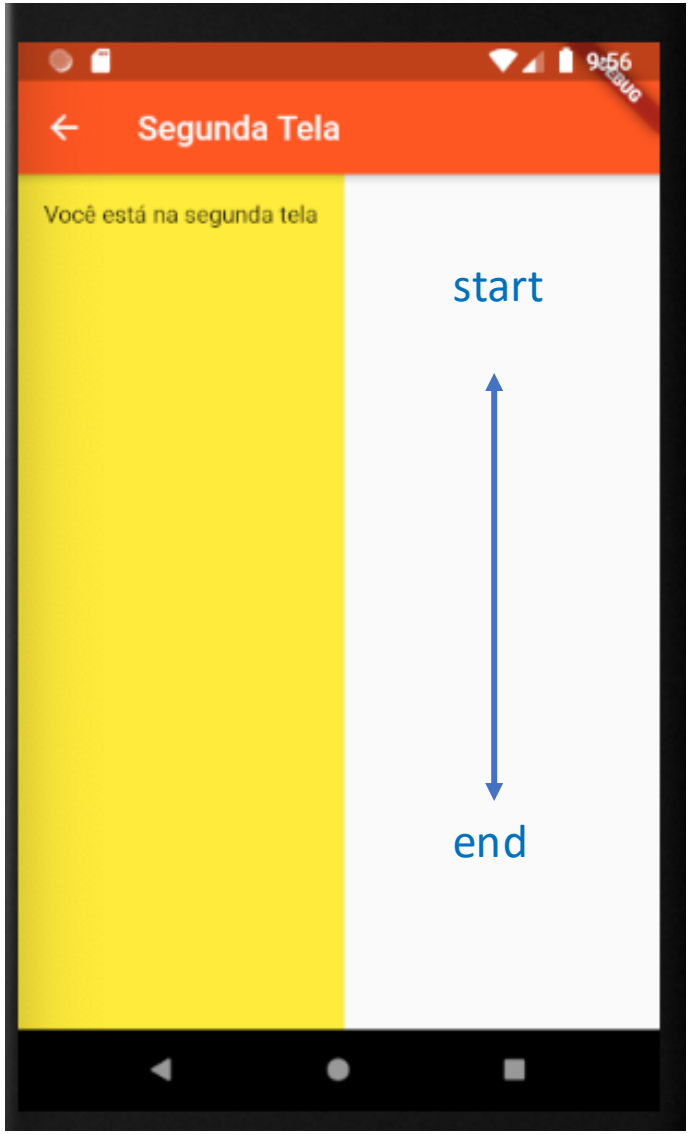


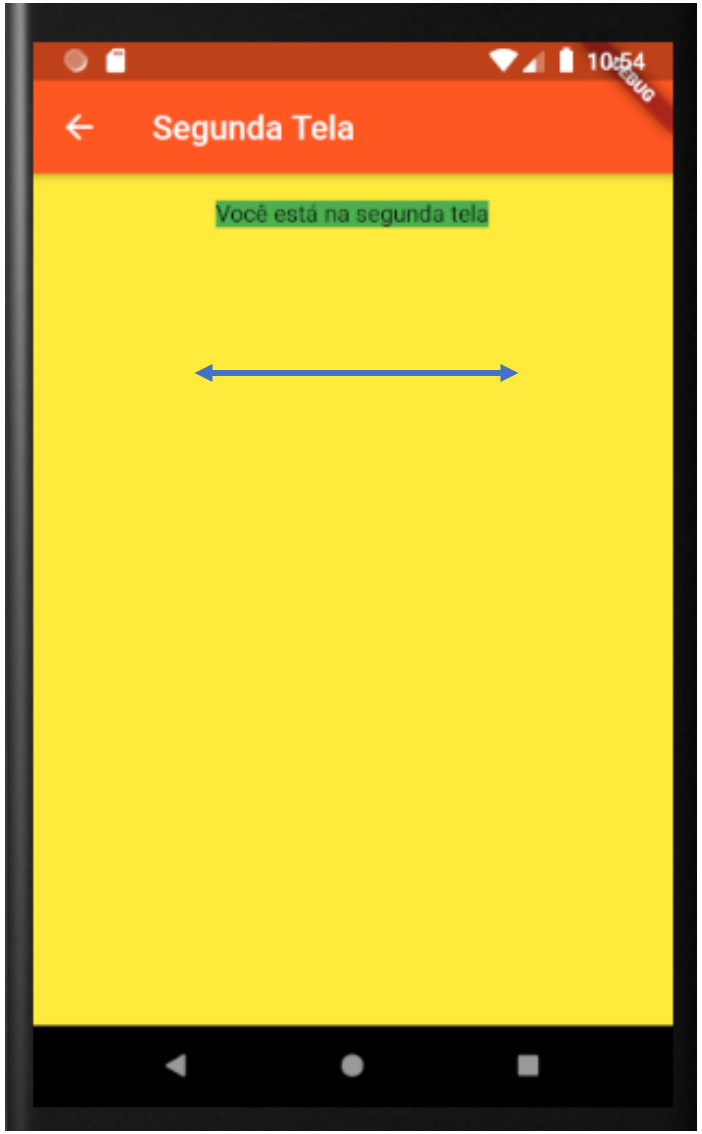
CrossAxisAlignment considera o eixo cruzado da linha, mas ele acaba dependendo do MainAxisAlignment, pois se não for indicado, é considerado start

Fonte: https://flutter.dev/docs/development/ui/layout

MainAxisAlignment *X* CrossAxisAlignment - Column







Fonte: https://flutter.dev/docs/development/ui/layout

Referências Bibliográficas

- Curso da Udemy – Flutter Essencial do professor Ricardo Lecheta.
- Curso da Udemy - Desenvolvimento Android e IOS com Flutter 2020 – Crie 15 Apps do professor Jamilton Damasceno.
- Site Flutter – flutter.dev